Rise Above Foundation Cebu

Optimizing Resource Allocation in Disaster Relief Operations through a Web-Based Donation System

PROJECT TEAM

DINAGUIT, SHEENA MARIE M.
DUAL, AUBRIE MARIE V.
ISIDRO, JAMES LLOYD P.
UTRERA, JULIE ANNE M.

Document Version:	*1.0*
Document Date:	SEPTEMBER 10, 2026

Revision History Page

Version	Date	Change Description	Author/s
1.0	*July 29, 2026*	*Full System Name*	*Lastname, Firstname* *Lastname, Firstname* *Lastname, Firstname*

Table of Contents

Project & System Overview 1

1. Summary 1

2. Project General Information 1

2.1. Stakeholders 1

2.2. Objective of the Project 1

2.3. Success Criteria 2

3. System General Information 2

3.1. System Architecture 2

3.2. User Feedback & UX Considerations 2

3.3. Business Flow 3

3.4. Permission Matrix 3

3.5. Database Metadata 4

3.6. Environments 4

3.7. Service Accounts 4

4. Functional Requirements 4

4.1. Core System Requirements 4

4.2. Governance 5

Implementation & Proposal Summary 8

1. Timeline & Milestones 8

2. Budget & Resource Requirements 8

3. Risk & Mitigation 9

4. Call to Action 9

Project & System Overview

1. Summary

The **Rise Above Foundation Cebu Donation Management System** was developed to help improve the management and distribution of donations during disaster relief operations. The system was created because manually managing donations, beneficiaries, volunteers, and relief requests can cause delays, difficulty in tracking resources, and problems in organizing information. The foundation also needs a **centralized database** to securely store and manage important data such as donor information, donations, inventory, volunteers, barangays, relief requests, and distribution records.

The system is a **web-based donation management system** that allows the foundation to record and monitor donations, manage data through a centralized database, organize volunteers and barangays, handle relief requests, monitor inventory, and track the distribution of resources. It helps the organization keep information organized and supports more efficient management of disaster relief operations and resource allocation.

2. Project General Information

2.1. Stakeholders

The following list outlines the components involved in this project:

2.1.1. Primary Stakeholders

Rise Above Foundation Cebu is a non-profit organization that provides assistance and support to communities affected by disasters and other difficult circumstances. The organization accepts and manages donations, coordinates relief activities, and distributes resources to communities in need. For this project, the foundation serves as the **client organization** and will provide the requirements, feedback, and validation needed for the development of the Web-Based Donation Management System.

The following are the points of contact within the organization/business who are involved in this project:

Name of POC	Email	Phone Number
Bartolome, Maria Jessa	maria.riseabove@gmail.com	(0998) 849 9363

2.1.2. End Users

Foundation Admin. The Foundation Admin will manage the overall operations of the system from the foundation's side. The Admin will manage staff accounts, donors, volunteers, barangays, donations, inventory, relief requests, distributions, and system records. The Admin will also review information and monitor the status of relief operations.

Staff. Staff members will use the system to assist with the day-to-day management of donations and relief operations. They will record and verify donations, manage inventory, process barangay assistance requests, update distribution records, and prepare basic reports based on their assigned responsibilities.

Donors. Donors will use the system to register their accounts and submit donation information. They can provide details about their monetary or in-kind donations and view the status or records of their submitted donations.

Volunteers. Volunteers will use the system to register and manage their volunteer information, view assigned tasks, record their activities or duty hours, and submit required proof of completed activities. This includes both intern volunteers and individual volunteers.

Barangay Representatives. Barangay Representatives will use the system to submit assistance requests on behalf of their barangay, monitor the status of requests, view approved distributions, and submit proof of received assistance when required. They can also review their previous assistance records.

2.2. Objective of the Project

The project aims to develop a centralized web-based donation management system that will help Rise Above Foundation Cebu manage donations, inventory, barangay assistance requests, volunteers, and relief distributions in an organized manner. The system aims to reduce manual record-keeping, improve the accuracy and accessibility of donation and

inventory records, make it easier to monitor relief resources, and support the foundation in allocating available resources based on approved assistance requests.

2.3. Success Criteria

The project will be considered successful upon the completion and evaluation of a fully functional web-based donation management system for Rise Above Foundation Cebu within the 2026–2027 academic year. The system will be tested and evaluated by the identified end users from the foundation to determine whether it meets their operational needs. Success will be measured through user acceptance testing based on the system's functionality, usability, reliability, efficiency, and security, as well as its ability to improve the management of donations, inventory, barangay assistance requests, volunteer records, and relief distributions. A positive outcome will be defined as the majority of evaluators rating the system “Acceptable” or higher across the evaluation criteria, confirming that the system can provide a more organized, accessible, and efficient approach to managing the foundation's relief operations.

3. System General Information

3.1. System Architecture

The proposed system will use a **three-layer web-based architecture** consisting of the presentation layer, application layer, and database layer. The presentation layer will use **React** to provide the user interface for the different system roles. The application layer will use **PHP** to process requests, perform system functions, and connect the front end to the database. The database layer will use **MySQL** to store and manage user accounts, donations, inventory, assistance requests, volunteer records, and distribution information. The system will be developed using **JavaScript, PHP, and SQL**, with **React and Vite** used for the front-end development. During development and deployment, the system will run through the appropriate web server environment and will connect the front end, PHP API, and MySQL database to allow users to access and manage information through the system.

Core Components:

- **Cloud Deployment Platform –** Used to host and provide access to the web-based system when deployed online.
- **Database Tool/Platform – MySQL –** Used to store and manage system data, including users, donations, inventory, assistance requests, volunteers, and distributions.

- **Other Platform – XAMPP –** Used during development to provide the local Apache web server and MySQL/MariaDB database environment.
- **Programming Language – JavaScript and PHP –** JavaScript will be used for the front-end functions and user interactions, while PHP will handle the system's back-end processes and API requests.
- **Framework – React with Vite –** React will be used to build the system's user interface and reusable components, while Vite will support the development and build process.

3.2. User Feedback & UX Considerations

User feedback is considered an important basis for improving the usability and effectiveness of the Web-Based Donation Management System. The attached document requires feedback to be considered before and after deployment; however, it does not provide documented survey results, interview transcripts, usability testing results, or analytics from an existing system. Therefore, the feedback findings described in this section are treated as scenario-based design considerations rather than measured user results. Pre-launch feedback may be obtained through direct user input such as surveys, interviews, and usability testing, which provide explicit and qualitative information about user needs and difficulties. Quantitative information may be obtained from the number or frequency of reported issues, while implicit feedback may be considered from observed user behavior during system use, where such information is available. These forms of feedback are relevant to the system because its users perform different activities based on their roles, including managing donations, inventory, assistance requests, distributions, volunteer activities, and request status tracking.

Four relevant feedback considerations are identified for the proposed system. A **Bug Report** may involve an incorrect or incomplete update of donation, inventory, or distribution records, which can affect the accuracy of relief operations. A **Usability Issue** may occur when authorized users have difficulty locating specific records, supporting the need for search and filtering functions. A **Feature Request** may involve Barangay Representatives requiring clear visibility of the progress of their assistance requests, which is addressed through request status tracking and status updates. A **Positive Feedback** may concern role-specific dashboards and access controls because these allow users to view information and functions relevant to their responsibilities. These considerations correspond directly with the system's

functional requirements for record management, search and filtering, request status tracking, dashboards, notifications, and role-based access.

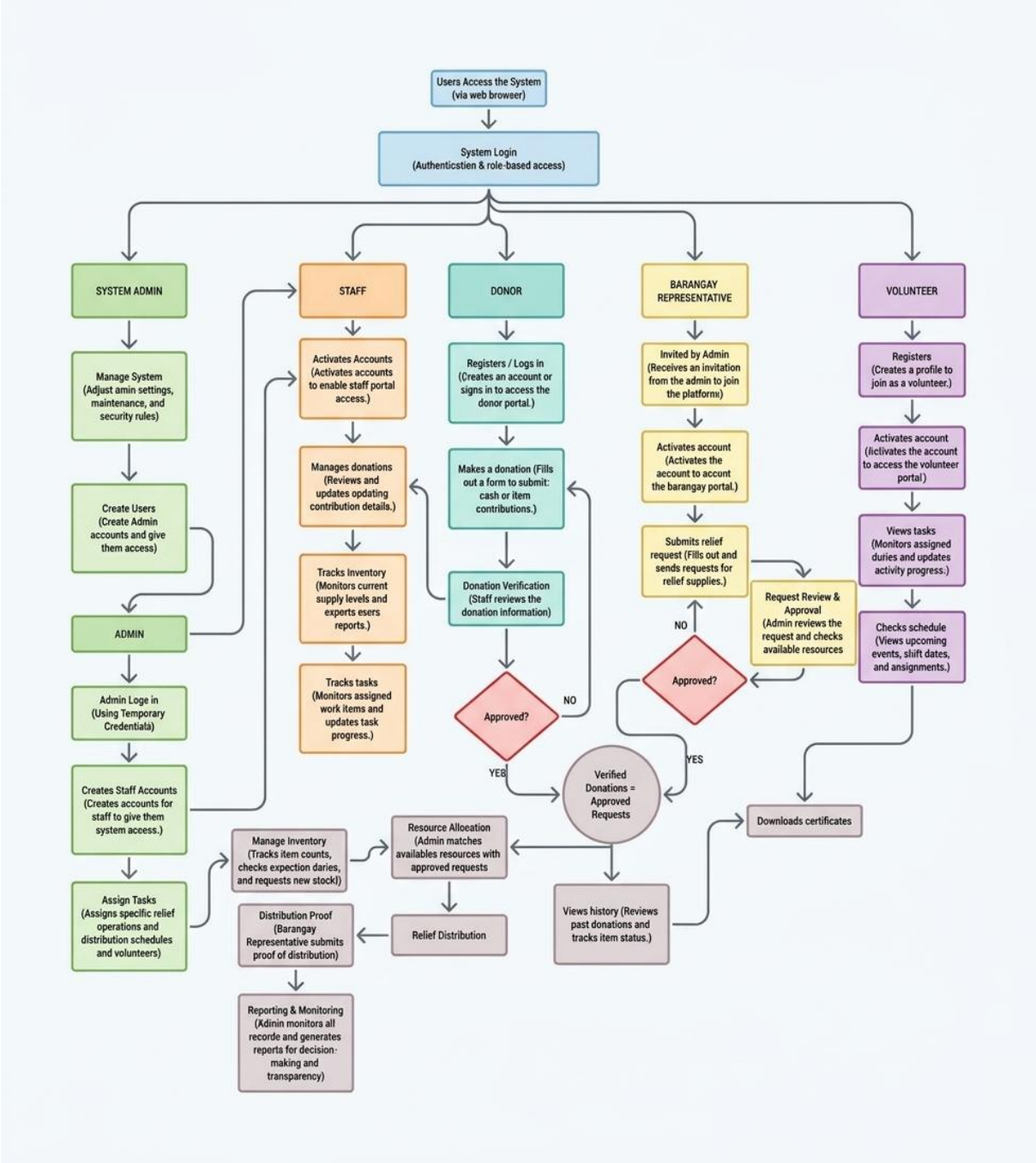
The feedback items are prioritized using the **MoSCoW framework**, with functions necessary for core disaster relief operations classified as Must Have. Accurate management of donations, inventory, assistance requests, and distributions receives the highest priority because errors or difficulties in these functions can directly affect resource monitoring and allocation. Usability improvements such as search and filtering and request status visibility are also prioritized because they support efficient access to operational information. Lower-priority enhancements may be considered after essential functions have been addressed, subject to their relevance to the system's objectives and available development effort. This approach ensures that feedback resulting in significant effects on operational accuracy, security, and user access is addressed before less critical improvements.

The resulting design decisions are aligned with the system's three-layer web architecture and functional requirements. The system uses a presentation layer for user interaction, an application layer for processing, and a database layer for storing system records. Feedback concerning record accuracy is therefore addressed through the functional requirements for donation, inventory, relief request, and distribution management, while usability concerns are addressed through dashboards, search and filtering, request status tracking, and notifications. Role-based access is retained to ensure that improvements to usability do not provide users with functions or information beyond their assigned responsibilities.

After deployment, feedback will continue to be collected through designated support channels and documented by support personnel until concerns are resolved. The system's support process already provides for reporting technical and system-related concerns to designated System Support personnel or the Super Admin, while the Full User Manual provides guidance for common functions and troubleshooting. Reported concerns are prioritized according to severity, with critical issues receiving immediate attention, moderate issues addressed as soon as possible, and minor concerns handled during regular support hours. Feedback and recurring concerns may be reviewed during regular system and governance reviews and when significant system changes, new features, or updated procedures are introduced.

To close the feedback loop, users should be informed when an accepted concern or improvement results in a system or procedural change. Communication should identify what was changed, the reason for the change, when it will take effect, and how it benefits the affected users or operations. Significant changes may also be supported by refresher training and updates to the Full User Manual. This process connects user feedback with prioritization, design decisions, implementation, and continuous improvement while maintaining consistency with the system's operational and governance requirements.

3.3. Business Flow



For each process (in separate paragraph), add explanation. Make sure that it is understandable and easy to read.

3.4. Permission Matrix

List of Permission on PRODUCTION environment:

Group Name	Type	Permission Level
DMS_SuperAdmin	*Super Admin*	*Full Control*
DMS_SystemAdmin	*System Administrator*	*Full Control*
DMS_ITManager	*IT Manager*	*Approve*
DMS_Staff	*Staff*	*Edit*
DMS_Volunteer	*Volunteer*	*Contribute*
DMS_Barangay	*Barangay Representative*	*Contribute*
DMS_Donor	*Donor*	*Contribute*

List of Permission on UAT environment:

Group Name	Type	Permission Level
DMS_SuperAdmin	*Super Admin*	*Full Control*
DMS_SystemAdmin	*System Administrator*	*Full Control*
DMS_ITManager	*IT Manager*	*Full Control*
DMS_Admin	*Foundation Administrator*	*Full Control*
DMS_Staff	*Staff*	*Edit*
DMS_Volunteer	*Volunteer*	*Contribute*
DMS_Barangay	*Barangay Representative*	*Contribute*
DMS_Donor	*Donor*	*Contribute*

List of Permission on TESTING environment:

Group Name	Type	Permission Level
DMS_SuperAdmin	*Super Admin*	*Full Control*
DMS_SystemAdmin	*System Administrator*	*Full Control*
DMS_ITManager	*IT Manager*	*Full Control*
DMS_Admin	*Foundation Administrator*	*Full Control*
DMS_Staff	*Staff*	*Edit*
DMS_Volunteer	*Volunteer*	*Edit*
DMS_Barangay	*Barangay Representative*	*Edit*
DMS_Donor	*Donor*	*Edit*

3.5. Database Metadata

The system utilizes a relational database management system hosted on TiDB Cloud (Serverless MySQL 8.0 compatible engine) operating with utf8mb4 character encoding and utf8mb4_general_ci collation. The complete schema comprises twenty-seven (27) normalized tables structured across user role-based access control, donation lifecycles, relief inventory tracking, kit repacking operations, calamity distribution logistics, volunteer shift scheduling, and cryptographically verifiable digital certificates.

Database Metadata and Data Dictionary:
Document File: database/DATABASE_METADATA.md (located in the project database folder)
GitHub Repository Path: github.com/jamis-ops/Donation-Management-System/blob/main/database/DATABASE_METADATA.md

Master SQL Schema:
Database File: database/donation_system.sql
GitHub Repository Path: github.com/jamis-ops/Donation-Management-System/blob/main/database/donation_system.sql

3.6. Environments

For the System:

Production	UAT	Test
https://donation-system-frontend.vercel.app (Primary Production SPA on Vercel Global Edge CDN)	https://donation-system-uat.vercel.app (UAT Staging Environment for Rise Above coordinators)	http://localhost:5173 (Local Vite and React Development Server)

For the API:

Production	UAT	Test
https://donation-system-api.onrender.com/api (Dockerized PHP 8.2 on Render Cloud, Singapore Region)	https://donation-system-api-uat.onrender.com/api (UAT Staging API Instance on Render Cloud)	http://localhost:8000/api (Local PHP Built-in Server CLI)

For the Database:

Production	UAT	Test
gateway01.ap-southeast-1.prod.aws.tidbcloud.com :4000 Database: donation_system (TLS/SSL Encrypted Serverless Cluster)	gateway01.ap-southeast-1.prod.aws.tidbcloud.com :4000 Database: donation_system_uat (Isolated UAT Staging Branch)	127.0.0.1:3306 or local_test.sqlite Database: donation_system_test (Local SQLite or MySQL Dev Instance)

3.7. Service Accounts

For the System:

Production	UAT	Test
Primary Super Admin: admin@riseabovefoundation.org Notification Mailer (Brevo): noreply@riseabovefoundation.org Cloud DevOps Infrastructure: devops@riseabovefoundation.org	UAT Lead Administrator: uat-admin@riseabovefoundation.org UAT Notification Mailer: uat-mailer@riseabovefoundation.org UAT QA Coordinator: uat-tester@riseabovefoundation.org	Local Development Superuser: test-admin@localhost.dev Mock Notification Mailer: mailhog@localhost.dev Capstone Lead Developer: jamesisidro0808@gmail.com

4. Functional Requirements

4.1. Core System Requirements

User Account and Authentication. The system shall allow authorized users to create accounts, log in, log out, and manage their profiles. Access to system features shall depend on the user's assigned role, such as Admin, Staff, Volunteer, Donor, and Barangay Representative.

Dashboard. The system shall provide dashboards that display important information based on the user's role. The dashboard may show donation records, relief requests, inventory status, assigned tasks, and other relevant activities.

People and Partner Management. The system shall allow the Admin to manage records of staff, donors, volunteers and interns, barangays, and partner organizations. Users shall be able to add, update, view, and manage the necessary information stored in the system.

Donation Management. The system shall allow donors to submit donation information and provide details about the donated items or resources. Admin and authorized staff shall be able to review, record, categorize, update, and monitor donations.

Inventory Management. The system shall allow authorized users to record, categorize, and monitor available relief goods and resources. The system shall update inventory records when items are received, assigned, distributed, or released.

Relief Request Management. The system shall allow Barangay Representatives to submit assistance requests for disaster-affected communities. Admin and authorized staff shall be able to review, approve, update, and monitor the status of each request.

Distribution Management. The system shall allow authorized users to record the distribution of donations and relief goods to approved beneficiaries or barangays. The system shall store information such as the items distributed, quantity, recipient, date, and distribution status.

Distribution Proof. The system shall allow authorized users to upload and store proof of distribution, such as photos or supporting documents. This will help provide records that the requested resources were properly distributed.

Volunteer and Task Management. The system shall allow volunteers and interns to register and submit their CV or resume. Admin or authorized staff shall be able to assign tasks, monitor volunteer activities, and update task status.

Request Status Tracking. The system shall allow Barangay Representatives to view the current status of their assistance requests. Status updates shall help users know whether a request is pending, approved, being processed, or completed.

Reports and Monitoring. The system shall provide records and reports related to donations, inventory, relief requests, distributions, volunteers, and other activities. These reports shall help the foundation monitor operations and make better decisions.

Notifications. The system shall provide notifications or status updates for important activities, such as new assistance requests, donation updates, task assignments, and changes in request status.

Search and Filtering. The system shall provide search and filtering functions to help authorized users quickly find specific records, such as donors, donations, inventory items, relief requests, volunteers, and distribution records.

Data Security and Access Control. The system shall protect stored information by restricting access to authorized users. Each user shall only be able to access features and information allowed by their assigned role.

Tech Specifications Needed for Users:

- A device such as a laptop, desktop computer, or tablet with internet access.
- A supported modern web browser, such as Google Chrome, Microsoft Edge, or Mozilla Firefox.
- A stable internet connection for accessing the system, uploading files, viewing records, and downloading documents.
- A device with sufficient storage for uploaded files, photos, and documents, when required by the system.

4.2. Governance

Governance is necessary for the Web-Based Donation Management System to establish clear responsibilities, approval processes, and decision-making authority throughout the system lifecycle. It provides a structured basis for managing system access, data, security, privacy, documentation, system changes, maintenance, and operational activities. Governance also helps ensure that system practices remain consistent with the objectives of Rise Above Foundation Cebu and with applicable requirements, including the Data Privacy Act of 2012, the organization's Acceptable Use Policy (AUP), and the Bring Your Own Device (BYOD) Policy.
The governance scope covers user access and permissions, data protection, security and privacy, system administration, documentation, technical maintenance, operational records, and the management of system-related concerns. Access is governed according to assigned roles so that users receive only the functions and information required for their responsibilities. The Super Admin manages system-level permissions, while the Foundation Admin reviews and confirms access appropriate to foundation responsibilities. Access is reviewed at least every six months and whenever a user's role or responsibilities change, with unnecessary permissions removed when they are no longer required.

Governance also establishes accountability among the personnel responsible for the system. The Super Admin serves as the System Owner and Access Manager, the IT Manager provides overall IT governance and technical oversight, and the System Administrator handles day-to-day technical administration, monitoring, maintenance, and troubleshooting. The Foundation Admin provides operational oversight for donations, assistance requests, inventory, volunteers, and distributions, while Staff support daily system operations and maintain the accuracy and completeness of operational records. Users such as Donors, Volunteers, and Barangay Representatives are responsible for providing accurate information and following their assigned permissions.

Standardization is supported through defined responsibilities, permission levels, security requirements, data classifications, and operational procedures. The governance plan ensures that access decisions are subject to appropriate approval, personal and operational information receives the required level of protection, and system concerns are directed to personnel with the appropriate authority. Reports and logs are also restricted to authorized users for monitoring and auditing purposes, supporting accountability while protecting system information.

Governance shall be reviewed as system requirements, risks, responsibilities, and operational needs change. Changes involving significant system functions, procedures, security concerns, or user responsibilities should be reviewed by the appropriate authority before implementation. Training and refresher sessions may also be conducted when significant system changes, new features, or updated procedures are introduced. Through regular access reviews, issue escalation, security and privacy requirements, and ongoing review of system procedures, governance supports the sustainable and accountable operation of the Web-Based Donation Management System.

4.2.1. Roles and Responsibilities

a. Super Admin

The **Super Admin** serves as the primary **System Owner and Access Manager** of the Web-Based Donation Management System. This role oversees user accounts, role assignments, permissions, and major system settings. The Super Admin ensures that access privileges are properly assigned and maintained according to each user's role. Access-related concerns, permission issues, and major system administration matters are referred to the Super Admin for appropriate action or escalation.

b. IT Manager

The **IT Manager** provides overall **IT governance and technical oversight** for the system. This role establishes technical direction, reviews security and maintenance concerns, and ensures that appropriate IT resources and support are available. The IT Manager also oversees major technical incidents, security concerns, and system issues that require management-level decisions. Matters beyond the authority of the System Administrator or Super Admin may be elevated to the IT Manager for resolution.

c. System Administrator

The **System Administrator** handles the **day-to-day technical administration and maintenance** of the system. This includes system configuration, user access administration based on approved permissions, system monitoring, and technical troubleshooting. The System Administrator also coordinates the resolution of system errors and technical incidents. Issues requiring major configuration changes, security decisions, or management approval are escalated to the Super Admin or IT Manager.

d. Foundation Admin

The **Foundation Admin** serves as the primary **operational approver** for Rise Above Foundation Cebu. This role manages and reviews donations, assistance requests, inventory activities, volunteer records, and resource distributions. The Foundation Admin ensures that operational records are properly reviewed and that activities follow the foundation's established procedures. Concerns involving foundation operations, records, or resource allocation are directed to this role for appropriate action.

e. Staff

The **Staff** supports the daily implementation of foundation operations within the system. Assigned tasks may include processing donation records, updating inventory, reviewing assistance requests, recording distributions, and maintaining other operational information. Staff members must observe their assigned permissions and maintain the accuracy and completeness of records. Issues beyond their access or authority are referred to the Foundation Admin or System Administrator, depending on the nature of the concern.

f. Donor

The **Donor** provides accurate personal and donation information when using the system. This role may submit donation details, monitor donation status, and maintain the information associated with the donor account. Donors must protect their account credentials and use the system only for authorized activities. Concerns related to donation submissions or account access may be raised through the designated support channel.

g. Volunteer

The **Volunteer** participates in assigned activities and uses the system to view tasks, record activities, and submit required proof or duty-hour information, when applicable. Volunteers are expected to provide complete and accurate activity records and comply with their assigned permissions. Concerns regarding assignments, rendered hours, or submitted proof are referred to the Foundation Admin or authorized Staff.

h. Barangay Representative

The **Barangay Representative** acts on behalf of the barangay in submitting assistance requests and providing the required information for resource allocation. This role may also review distributions and submit supporting proof or documentation through the system. The Barangay Representative must ensure that submitted requests and records are accurate and complete. Concerns involving assistance requests or distributions are coordinated with the Foundation Admin or authorized Staff.

4.2.2. Code Naming Convention

The establishment of uniform code naming conventions ensures maintainability, seamless readability, and cross-stack consistency across a heterogeneous architecture combining a modern React Single Page Application (SPA) frontend, a PHP 8.2 RESTful backend API, and a relational MySQL/TiDB database. Consistent naming prevents defects, eases collaborative development among team members, and lowers the cognitive load when inspecting distributed system components.

- **PascalCase:** Exclusively applied to React UI Components (e.g., StatsSection.jsx, PartnersSection.jsx, AdminLayout.jsx), React Context Providers (AuthContext.jsx, ThemeContext.jsx), and PHP Object-Oriented Classes or Exceptions.
- **camelCase:** Applied to JavaScript variables, React custom hooks (e.g., useApiList, usePagination, useAuth), component props, JavaScript helper functions (e.g., fetchDonationStats(), handleVerify()), and JSON API request or response payload attributes.
- **snake_case:** Applied to all database tables and column identifiers (e.g., inventory_items, tracking_code, proof_file, created_at) and backend PHP variables,

database query parameters, and procedural helper scripts (e.g., $db_host, get_db_connection()).

• **UPPER_SNAKE_CASE:** Strictly reserved for global constants, configuration definitions, and environment variables (e.g., DB_HOST, DB_PASSWORD, JWT_SECRET, VITE_API_URL, MAX_UPLOAD_SIZE_MB).

• **kebab-case:** Applied to CSS class names (e.g., .stat-card, .btn-primary, .modal-backdrop), URL web route paths (e.g., /admin/inventory-management, /portals/donor/certificates), and REST API file and resource endpoints.

Prefixes and Suffixes Requirements:

• **Booleans:** Must begin with affirmative verbs such as is, has, can, or should (e.g., isLoading, isVerified, hasPermission, canDistribute).

• **React Custom Hooks:** Must invariably start with the use prefix in compliance with React specifications (e.g., useApiList, usePagination, useAuth).

• **Event Handlers:** Must use handle for function definitions (e.g., handleDonationSubmit) and on for passed callback props (e.g., onSubmit, onStatusChange).

• **Foreign Keys:** Must always be suffixed with _id matching the referenced singular entity (e.g., user_id, donor_id, role_id).

Rules for Abbreviations: Cryptic abbreviations are strictly prohibited. Standard industry abbreviations are permitted: id, db, api, url, cv, auth, lgu. Business domain entities must use full descriptive terms (e.g., use beneficiary instead of ben, and repacking_job instead of rpk_j).

Exceptions: Environment variables and constants are written in ALL_CAPS. Embedded acronyms in camelCase maintain consistent casing (e.g., parseCsvFile, fetchApiData).

4.2.3. Coding Standards

Overall Coding Philosophy: The development team adheres to core software engineering methodologies: 1) DRY (Don't Repeat Yourself) — common database routines, authentication filters, and UI notification modals are centralized into reusable modules; 2) SOLID Principles — especially Single Responsibility for API endpoints and Open/Closed for relief status workflows; 3) KISS (Keep It Simple, Stupid) — avoiding speculative over-engineering in favor of robust, transparent logic; and 4) Defensive Programming — rigorous server-side input sanitization, strict parameter type-casting, and prepared PDO SQL statements to block SQL injection and Cross-Site Scripting (XSS).

• **Frontend Style Guide:** Airbnb JavaScript Style Guide and Official React.dev best practices.

• **Backend Style Guide:** PHP-FIG PSR-12 (Extended Coding Style Guide) and PSR-4 (Autoloading Standards).

• **Database Style Guide:** SQL Style Guide (ISO/IEC 9075) with capitalized standard SQL keywords and lowercase table/column identifiers.

Formatting Rules: JavaScript, JSX, and CSS use 2-space soft indentation; PHP backend code uses 4-space indentation. Maximum line length is capped at 100 characters. One True Brace Style (1TBS) is enforced across JavaScript; PSR-12 brace placement is followed in PHP (classes and methods open on next line; control structures open on same line). Semicolons are mandatory.

Commenting and Documentation Expectations: All exported functions, hooks, and endpoints must be documented using structured JSDoc (Frontend) or PHPDoc (Backend) indicating parameter data types, expected returns, and exceptions thrown. Inline comments must document architectural intent (why) rather than self-evident code statements (what).

Error Handling Conventions: All API responses conform to a unified JSON response schema containing success (boolean), data (payload or null), message (descriptive string), and error (standardized error code or null). Explicit HTTP status codes must be returned (200, 201, 400, 401, 403, 404, 422, 500). Internal database errors are logged securely server-side without leaking raw credentials or stack traces to client browsers.

Tools Used to Enforce Standards: ESLint and Prettier for frontend formatting; PHP_CodeSniffer (phpcs) for backend PSR-12 validation; Git pre-commit hooks configured via Husky to block non-compliant commits.

4.2.4. Environment Variables Standards

Purpose: In strict conformance with the Twelve-Factor App methodology (Config III), all environment-specific configurations, cryptographic secrets, and third-party credentials are completely decoupled from application source code, preventing accidental credential leaks and allowing identical builds to run across Development, UAT, and Production.

Naming Conventions and Categories: All environment variables use strict UPPER_SNAKE_CASE prefixed by domain:

- **Database:** DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DB_SSL.
- **Integrations:** BREVO_API_KEY, GEMINI_API_KEY, MAIL_FROM_EMAIL, MAIL_FROM_NAME.
- **Frontend (Vite):** VITE_API_URL, VITE_APP_ENV, VITE_APP_VERSION.
- **CORS and Security:** FRONTEND_URL, SESSION_LIFETIME, APP_SECRET.

Storage and Locations: Locally, environment variables are loaded from an uncommitted .env file. In cloud environments, secrets are managed securely through the Render Environment Dashboard (Backend) and Vercel Project Settings (Frontend). A sanitized template file (.env.example) is committed to version control with descriptive placeholders.

Hardcoding Policies: Never hardcode passwords, API keys, database URLs, or security secrets. Permissible hardcoded items are limited to non-sensitive defaults, such as default pagination size (10 items), UI label constants, and fixed navigation routes.

Security Practices: .env and .env.local are permanently registered in .gitignore. Production secrets are restricted to designated system administrators. Secrets are rotated every 90 days or immediately upon team member offboarding.

4.2.5. Folder Structure

Organizing Principle: The repository employs a Layered, Decoupled Client-Server Modular Architecture. The codebase cleanly separates the user interface Single Page Application (/Frontend) from the stateless REST API (/Backend) and shared data migration assets (/database).

Top-Level Structure and Sub-Folder Responsibilities:

- **/Backend/api/:** PHP RESTful API endpoints (donations, inventory, distributions, auth), session handling, and CORS middleware.
- **/Backend/api/uploads/:** Secure file system directory for recipient proof photos, donation receipts, and volunteer CV uploads.
- **/Frontend/src/pages/:** Top-level page views (Home, AboutUs, FAQ, Public Donation Tracking, Login, Register).
- **/Frontend/src/portals/:** Role-based user portal layouts and specialized sub-views (Admin, Donor, Volunteer, Beneficiary, Staff).
- **/Frontend/src/components/:** Reusable modular interface components divided into /admin, /layout, /shared, and /home.
- **/Frontend/src/context/:** Global React Context providers (AuthContext.jsx, ThemeContext.jsx).
- **/Frontend/src/hooks/:** Reusable custom React hooks (useApiList.js, usePagination.js).
- **/Frontend/src/styles/:** Global CSS stylesheets, CSS custom property design tokens, and role-specific portal styles.
- **/database/:** Master SQL schema files (donation_system.sql), seed fixtures, and DATABASE_METADATA.md documentation.
- **/mail-service/:** Express / Node.js background mail notification microservice for queueing transactional notices.

Nested Rules and Naming: Components are grouped by feature domain. Any UI element reused across two or more portals must reside under /components/shared/. Folder names are lowercase with hyphens for services and PascalCase for React component folders.

4.2.6. Taxonomy/Metadata Standards

Purpose: Establishing rigorous taxonomy and metadata standards guarantees consistent data classification, predictable API contracts, structured code documentation, and unambiguous semantic versioning across the entire humanitarian relief system lifecycle.

• **Database Schema Taxonomy:** Tables are pluralized snake_case nouns (e.g., assistance_requests). Primary keys are consistently named id (BIGINT UNSIGNED AUTO_INCREMENT). Foreign keys strictly follow singular_parent_table_id. Audit timestamps created_at and updated_at are mandatory on all core transactional tables.

• **API Metadata and Envelope:** Every API endpoint returns explicit metadata headers (Content-Type: application/json; charset=utf-8, X-API-Version: 1.0.0). Paginated endpoints return metadata envelopes containing total_records, current_page, per_page, and total_pages.

• **Code Documentation Metadata:** Standardized PHPDoc and JSDoc annotations (@param, @returns, @throws, @deprecated, @author) must precede all core utility routines and custom hooks.

• **Version Control Tagging:** Semantic Versioning (SemVer 2.0.0) format (vMAJOR.MINOR.PATCH) is enforced (e.g., v1.0.0 Initial Capstone Release). Git commit messages follow Conventional Commits (feat:, fix:, docs:, refactor:, chore:).

• **Feature and Environment Flags:** Runtime flags (e.g., FEATURE_AI_VOLUNTEER_MATCHING=true, MAINTENANCE_MODE=false) dictate module behavior without code modifications.

4.2.7. Deployment Standards

Deployment Environments: 1) Development (Local machine localhost:5173 / localhost:8000) for rapid feature iteration; 2) UAT (Staging on Vercel and Render) for validation by Rise Above Foundation Cebu staff and SWU faculty panelists; and 3) Production (Vercel Global CDN + Render Docker + TiDB Cloud Serverless) for live humanitarian operations.

CI/CD Deployment Process and Pipeline:

• **1. Code Commit:** Developer pushes feature commits to GitHub repository branches.

• **2. Automated Trigger:** GitHub webhook notifies Render and Vercel build pipelines.

• **3. Build and Test:** Vercel executes 'npm run build' (Vite bundle optimization and lint validation); Render builds Docker container with PHP 8.2 and Apache.

• **4. Health Check Verification:** Render verifies zero-downtime container readiness at /api/health.php before shifting live traffic.

• **5. Database Migrations:** Schema modifications are applied using transaction-safe migration scripts against TiDB Cloud.

Branching Strategy: A structured GitFlow model is implemented:

• **feature/*:** Short-lived feature branches branched from develop.

• **develop:** Integration branch continuously deployed to the UAT / Staging environment.

• **main:** Production-ready protected branch. Pushing or merging directly to main is restricted.

• **hotfix/*:** Emergency patches branched from main and back-merged to develop.

Approval and Review Requirements: Merging into main requires a formal Pull Request (PR) with at least two peer reviews from capstone project members and validation that automated build checks succeed. Final approval is signed off by the Lead Developer.

Rollback Procedures:

• **Frontend (Vercel):** Access Vercel Dashboard -> Deployments -> locate previous verified deployment -> click 'Rollback to this Deployment'. Rollback executes instantaneously within 15 seconds.

• **Backend (Render):** Access Render Dashboard -> Service Activity -> select prior working Docker image build -> trigger redeploy.

• **Database (TiDB Cloud):** Point-In-Time Recovery (PITR) snapshots allow restoring the database state to any exact second prior to the failed deployment.

4.2.8. Permissions & Access Management

Access to the Web-Based Donation Management System will be defined according to the user's assigned role and the **Permission Levels table**. Each user will only be given access to the functions and information required for their responsibilities. The **Super Admin** will have the highest level of system access, while the **Foundation Admin, Staff, Volunteer, Barangay Representative, and Donor** will have permissions based on their assigned roles.

New access will be requested when a new user joins the organization or when an existing user requires a change in permissions due to a change in responsibilities. The **Foundation Admin** will review and approve access requests for foundation users, while the **Super Admin** will manage the corresponding system-level access. Users will not be given additional permissions without proper approval.

The **Super Admin** will be responsible for granting, changing, and revoking system permissions based on the approved user role. The Foundation Admin will be responsible for confirming that the requested access is appropriate for the user's responsibilities. Access will be reviewed **at least once every six months** and whenever there is a change in a user's role or responsibilities. The review will check whether users still require their current permissions and whether any unnecessary access should be removed.

When a user leaves the organization, completes their volunteer assignment, or changes to a different role, their access will be reviewed immediately. Unnecessary permissions will be removed, and the account may be deactivated

when the user no longer requires system access. This process will help prevent former or unauthorized users from accessing system information.

4.2.9. Security & Compliance

The Web-Based Donation Management System will follow applicable security and data privacy requirements, particularly the **Data Privacy Act of 2012 (Republic Act No. 10173)**, as well as the organization's **Acceptable Use Policy (AUP)** and **Bring Your Own Device (BYOD) Policy**. These policies will guide users in the proper use of the system, protection of confidential information, account security, and responsible use of personal devices when accessing the system.

AUP: Donation Management System AUP

BYOD Policy: Donation Management System BYOD

Data Classification Table

The Web-Based Donation Management System will collect and process personal and operational information necessary for the administration of donations and disaster relief operations. The following table classifies the collected data according to its nature and specifies the corresponding level of protection required to maintain confidentiality, integrity, and appropriate access.

Data Collected	Classification	Protection Level Required
Full Name	Personal Data	**Moderate** – Encrypt in storage and restrict access to authorized users only.
Email Address	Personal Data	**Moderate** – Protect using secure authentication and encryption.
Mobile Number	Personal Data	**Moderate** – Restrict access and validate

		input to prevent unauthorized disclosure.
Home/Office Address	Personal Data	**Moderate** – Store securely and allow access only to authorized personnel.
Username	Personal Data	**Moderate** – Protect from unauthorized access and modification.
Password (Hashed)	Personal Data	**High** – Store only as hashed values and never display or save in plain text.
User Role (Admin, Staff, Volunteer, Donor, Barangay Representative)	Personal Data	**Low to Moderate** – Restrict access to administrators.
Company/Organization Name	Personal Data	**Moderate** – Secure storage and limited access.
Barangay Representative Name	Personal Data	**Moderate** – Accessible only to authorized users managing requests.
Beneficiary Name	Personal Data	**Moderate –** The information shall be securely stored and restricted to authorized users involved in assistance processing and relief operations.
Beneficiary/Household Address	Personal Data	**Moderate –** Access shall be restricted to authorized personnel, and the

		information shall be protected against unauthorized disclosure.
Household Size	Personal Data	**Moderate –** The information shall be securely stored and accessible only to authorized personnel responsible for assistance assessment and resource allocation.
Donation Records	Personal Data	**Moderate** – Maintain confidentiality and prevent unauthorized modification.
Donation History	Personal Data	**Moderate** – Allow viewing only by authorized users or the donor concerned.
Inventory Records	Non-Personal Organizational Data	**Standard** – Protect against unauthorized editing and data loss.
Assistance Requests	Personal Data	**Moderate** – Restrict access to authorized administrators and staff.
Distribution Proof (Photos/Documents)	Personal Data	**High** – Store securely and limit access because files may contain identifiable individuals.
Reports and Logs	Personal Data	**Moderate** – Allow

		access only to authorized users for monitoring and auditing purposes.

4.2.10. Monitoring & Auditing

The Web-Based Donation Management System will monitor system activities and records to support security, accountability, and proper system operation. Monitoring will focus on user access, account activities, changes to important records, system errors, and security-related events. Particular attention will be given to activities involving donations, inventory, assistance requests, distributions, user accounts, and other records containing personal or operational information.

System activities and relevant changes will be recorded through system logs and audit records where supported by the application. These records may include user activity, date and time of activity, accessed or modified records, and other relevant information needed to review system use. System and database logs may also be reviewed to identify errors or unusual activities.

Routine monitoring will be conducted by the Super Admin, with the Foundation Admin responsible for reviewing activities related to the foundation's operations. Access and activity records will be reviewed at least once every six months, or more frequently when a security concern, system issue, or suspected unauthorized activity is reported.

When an unusual activity, unauthorized access attempt, data inconsistency, or system issue is identified, the responsible personnel will document the incident and assess its severity. Appropriate actions may include verifying the affected records, correcting inaccurate information, restricting or revoking user access, investigating the activity, and escalating serious incidents to the Super Admin and Foundation Admin. Security incidents involving personal data will be handled in accordance with the organization's security procedures and applicable data privacy requirements.

4.2.11. Training & Support

New users of the Web-Based Donation Management System will undergo an onboarding process based on their assigned roles. The onboarding process will include an introduction to the system, account and login procedures, role-specific functions, and basic security practices. Users will be guided through the functions relevant to their responsibilities and will be provided with the **Full User Manual** for reference.

Full User Manual: https://canva.link/p5p9r9ew66uuxgm

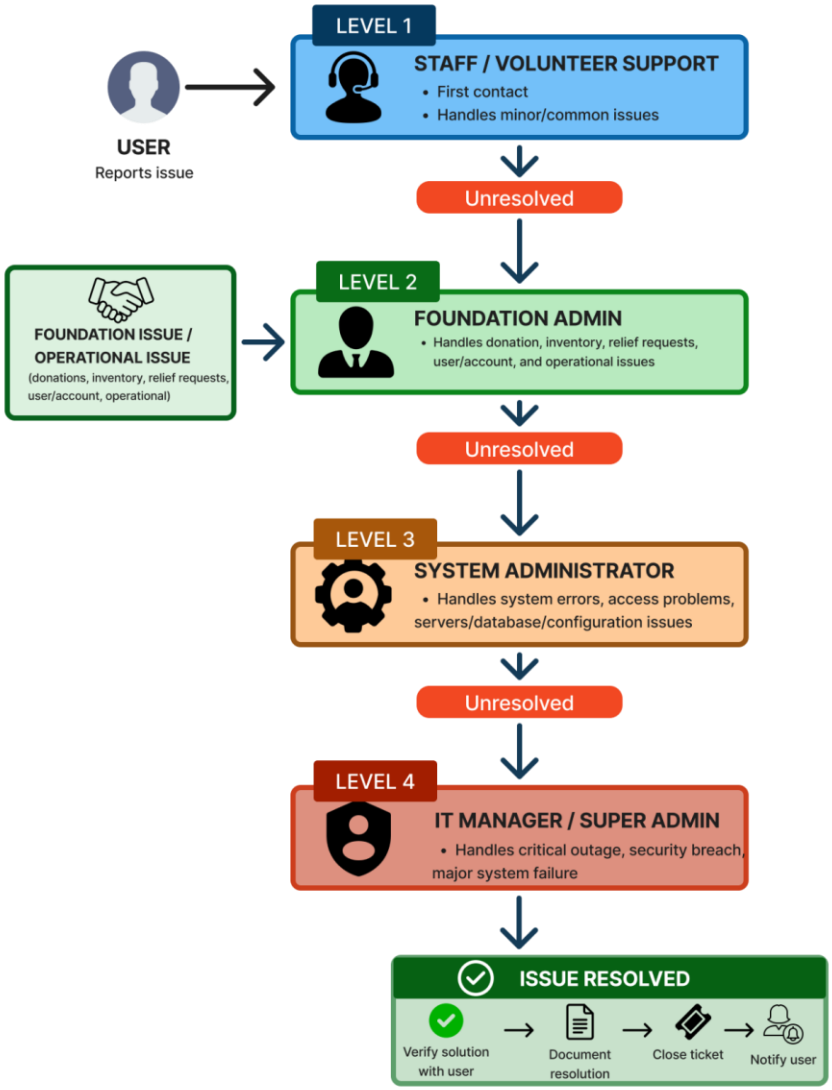
Training and refresher sessions may be conducted when significant system changes, new features, or updated procedures are introduced. Users will also be reminded of proper account security, data privacy, and appropriate system use when necessary.

Technical and system-related concerns may be reported to the designated **System Support** personnel or the **Super Admin**. Users may also refer to the Full User Manual for instructions on common system functions and troubleshooting. Since the system does not include a built-in ticketing feature, reported concerns will be documented and monitored by the designated support personnel until resolved.

Support requests will be prioritized according to their severity. **Critical issues** that prevent the system from being used will receive immediate attention, while **moderate issues** affecting specific functions will be addressed as soon as possible. **Minor issues** and general inquiries will be handled during regular support hours.

Self-service resources will be available through the **Full User Manual**, which will contain role-specific instructions, system procedures, troubleshooting steps, and other relevant guidance. Users are encouraged to consult the manual before requesting assistance for common or basic system concerns.

4.2.12. Escalation Path



Escalation Details

The table outlines the support escalation process for addressing system-related concerns. It specifies the responsible personnel at each escalation level, the conditions requiring escalation, the target resolution time, and the designated communication channels. This structured approach ensures that concerns are promptly directed to the appropriate personnel, facilitating efficient issue resolution and minimizing disruptions to system operations.

Level	Responsible Role	When to Escalate	Target Resolution Time	Communication Channel
Level 1	Staff / Volunteer Support	Login issues, forgotten password, navigation questions, basic user assistance	Same business day	Phone, Email, System Ticket
Level 2	Foundation Admin	Incorrect donation records, inventory issues, assistance request approval issues, volunteer assignment problems	Within 1 business day	System Ticket, Email, Phone
Level 3	System Administrator	Database errors, server downtime, permission issues, failed reports, systems bugs	Within 2–3 business days	System Ticket, Email
Level 4	IT Manager / Super Admin	Major system outage, security	Priority handling; resolution	Emergency Phone, Email, Incident Report

		breach, data loss, disaster recovery, critical technical failures	depends on severity	

Implementation & Proposal Summary

1. Timeline & Milestones

Phase	Duration / Target Date	Key Deliverable
Project Initiation	January – February 2026	The initial project scope, objectives, and proposed donation management system will be defined and documented based on the identified needs of Rise Above Foundation Cebu.
Requirements Gathering & Initial Planning	February – March 2026	The initial functional requirements, user roles, business processes, and system requirements were identified and documented during the planning phase.
Initial System Design	March – April 2026	The preliminary database structure, system architecture, business flow, and interface prototypes were prepared based on the initial requirements.
Defense & Scope Revision	April 2026	The initial system design and scope were presented during the defense. Panel feedback

		was used to identify the required changes and revisions to the system.
Revised Planning & Design	April 2026	The panel's recommendations from the defense were incorporated into the system requirements, user roles, business processes, database design, and interface designs. The revised planning and design were finalized at the end of April 2026.
System Development	June – August 2026	The core system modules, including authentication, role-based access, donation management, inventory, assistance requests, volunteer management, distributions, and reports, were developed and integrated into a functional web-based system.
System Testing & Debugging	July – August 2026	The developed modules were tested for functionality, and identified errors and system issues were addressed through necessary corrections and improvements during the testing phase.
System Refinement & Continued Testing	August – September 2026	System features and interfaces were refined based on testing results, consultations, and feedback, and the

		improvements were completed in September 2026.
User Acceptance Testing & Deployment	September 2026	The refined system was prepared for user acceptance testing and evaluated by selected Rise Above Foundation Cebu users, with evaluation results and feedback documented during the testing period.
Final Documentation & Project Closure	September – October 2026	The Full User Manual, system documentation, testing results, governance documents, and final revisions were completed and prepared for final presentation in October 2026.

2. Budget & Resource Requirements

In a table form, provide a cost breakdown and list what people/tools/materials are needed to execute this project.

What to include:

- *Cost breakdown — by category (e.g., software/licensing, hardware, training, labor/manpower)*
- *Team/resource requirements — roles needed and time commitment (e.g., 1 developer, 20 hrs/week, 3 months)*
- *Financial justification — briefly connect the cost to the value/savings it produces (e.g., "₱50,000 cost vs. ₱500,000 potential loss avoided"), since decision-makers need to see the ROI, not just the price tag*

3. Risk & Mitigation

In a table form, identify the key risks to this project/system and how each will be addressed.

What to include:

- *Risk — what could go wrong (e.g., budget overrun, staff resistance, technical failure, missed deadline, security gap)*
- *Likelihood/Impact — rough sense of how likely it is and how bad it would be if it happened*
- *Mitigation — the specific action that reduces or handles the risk*

Risk	Likelihood	Impact	Mitigation
e.g., Staff resistance to new system	*Medium*	*Medium*	*Phased rollout + training sessions before go-live*
e.g., Budget overrun	*Low*	*High*	*10% contingency built into budget; phased approach allows scope adjustment*

4. Call to Action

In 1–2 short paragraphs, close the document with a direct, specific request — this is the "ask."

What to include:

- *A one-sentence restatement of the core problem and proposed solution*
- *The specific decision or approval being requested (e.g., "We are requesting approval to proceed with Phase 1 by [date]")*
- *A clear next step for the reader (what happens if they say yes)*
- *Optional: a short line reinforcing urgency or value (e.g., cost of inaction vs. benefit of approval)*